# Java Object-Oriented Programming Project

Smart Task & Resource Management System (STRMS)

E3e_S6_Spring 2026

## Contents


**1 Introduction** **3**

**2 Learning Objectives** **3**

**3 Task Management System Overview** **3**
- 3.1 Task Module . . . 4
- 3.2 TaskManager . . . 5
- 3.3 System Components and User Roles . . . 5

**4 Class Structure** **7**
- 4.1 User Classes . . . 7
- 4.2 Task Related Classes . . . 7
- 4.3 Enumerations . . . 8
  - 4.3.1 TaskStatus . . . 8
  - 4.3.2 PriorityLevel . . . 8
  - 4.3.3 TaskCategory . . . 8
  - 4.3.4 NotificationType . . . 9
- 4.4 Utility Classes . . . 9

**5 Data Structures** **9**
- 5.1 ArrayList . . . 9
- 5.2 HashMap . . . 10
- 5.3 HashSet . . . 10
- 5.4 PriorityQueue . . . 10

**6 Task Dependencies and History** **10**
- 6.1 Task Dependencies . . . 10
  - 6.1.1 Dependency Rules . . . 10
  - 6.1.2 Circular Dependency Detection . . . 11
  - 6.1.3 Dependency-Based Task Activation . . . 11
- 6.2 Task History Tracking . . . 11
  - 6.2.1 Logged Events . . . 11
  - 6.2.2 History Storage and Access . . . 12
  - 6.2.3 Design Guidelines and Evaluation Criteria . . . 12
- 6.3 Task Dependencies & Circular Dependency Detection . . . 12
  - 6.3.1 Example of Linear and Circular Dependencies . . . 12
  - 6.3.2 Example of Valid Task Dependency Scenario . . . 14
  - 6.3.3 Example of Logging a Dependency Violation in History . . . 14

**7 JUnit Testing Requirements** **15**
7.1 JUnit Test Suite . . . . . 15
7.2 Mandatory Test Scenarios . . . . . 15
7.3 Testing Constraints . . . . . 15

**8 Custom Exceptions** **16**

**9 UML Guidelines** **16**

**10 Implementation Phases** **17**

**11 Project Instructions** **17**

# 1 Introduction

The goal of this project is to help students build solid skills in **Java object-oriented programming (OOP)**, **data structures**, **file I/O**, **exception handling**, **unit testing**, and **UML modeling**. The system, Smart Task & Resource Management System (STRMS), simulates real-world task management in an engineering environment.

This project includes:

- **Task dependencies**: tasks may depend on other tasks being completed first.
- **Task history**: all changes in task status and assignments are recorded.
- **Priority management**: tasks are processed according to their priority, ensuring that all dependencies are completed first.
- **Notifications**: console alerts inform users about task updates and changes.
- **Dashboard**: provides an overview of task statistics for users (e.g., task progress, status, priority, and assignments).

# 2 Learning Objectives

By completing this project, students will:

1. Apply the four pillars of object-oriented programming: inheritance, polymorphism, abstraction, and encapsulation.
2. Design and interpret UML diagrams (e.g., class and sequence diagrams).
3. Implement abstract classes and interfaces.
4. Use *enums* to ensure strong typing.
5. Work with Java collections (ArrayList, HashMap, HashSet, PriorityQueue).
6. Implement custom exceptions and perform proper error handling.
7. Read data and write them to files.
8. Manage task dependencies and maintain task history.
9. Implement unit and integration tests using *JUnit*.
10. Organize and structure a professional Java project.

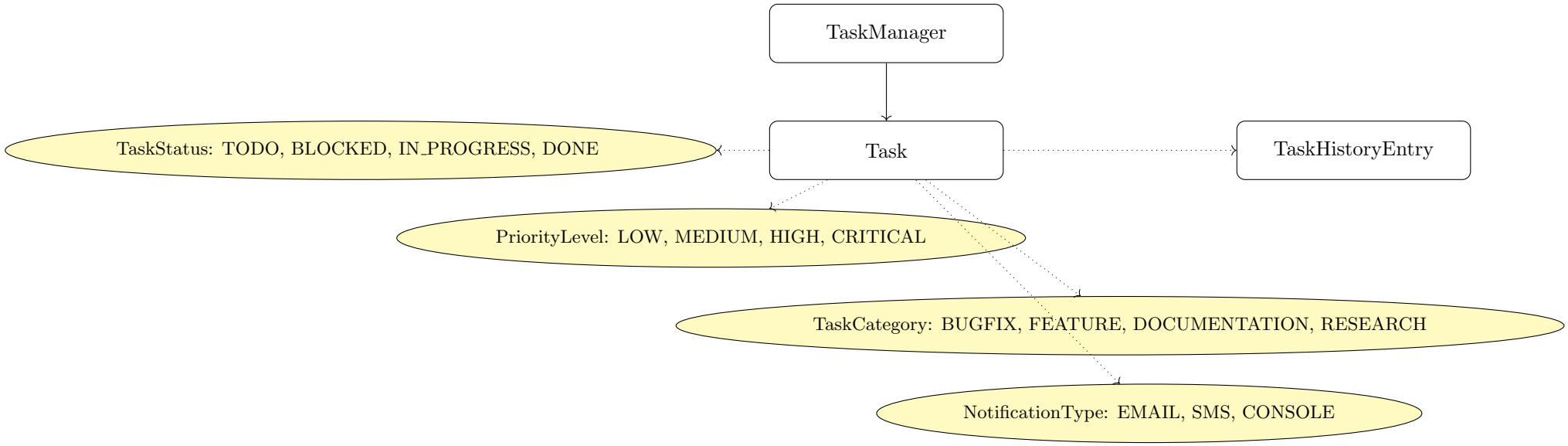
Students are required to respect the instructions and carry out all required tasks.

# 3 Task Management System Overview

The STRMS allows to:

- Create users with specific roles (Admin, Manager, Engineer).
- Create, assign, update, and delete tasks.
- Track task progress and dependencies.
- Maintain a history of tasks for auditing purposes.
- Prioritize tasks and manage their execution order.
- Store system data in files.
- Generate reports and display dashboards.

## 3.1 Task Module




- Each **Task**:
  - Encapsulates its own data (ID, title, assigned engineer, etc.) and behavior.
  - Uses *enumerations* to represent its internal state.
  - Maintains a list of tasks on which it depends.
  - Maintains a history of all actions performed on it.
  - Provides methods to update its state and append to its history.
  - Supports comparison with other tasks according to their priority.
- The **TaskManager** represents the system and is responsible for controlling task operations as well as enforcing rules on tasks. Therefore, it acts as the central controller of the system, enforcing business rules, maintaining data consistency, and managing all tasks.

## 3.2 TaskManager

The **TaskManager** is the central component of the STRMS system, responsible for managing all tasks and enforcing the system's rules. It encapsulates the operations needed to create, update, assign, and delete tasks while maintaining task dependencies. It also handles exceptions related to invalid operations, such as circular dependencies, incomplete prerequisites, or unauthorized actions. Additionally, it manages persistence by saving and loading task data and provides interfaces for generating reports, dashboards, and notifications.

Thus, the **TaskManager**, which represents the system, is responsible for controlling task operations and enforcing rules on tasks. Consequently, it may throw several exceptions, including the following:

- **CircularDependencyException:** Thrown when a circular dependency is detected among tasks, requiring knowledge of the global task dependency graph.
- **DependencyNotCompletedException:** Thrown when attempting to start or update a task whose prerequisite tasks have not yet been completed.
- **TaskNotFoundException:** Thrown when a requested task identifier does not exist in the system.
- **InvalidTaskStateException:** Thrown when a task state transition violates the business rules enforced by the `TaskManager`.
- **FilePersistenceException:** Thrown when reading from or writing to files fails.
- **InvalidRoleException:** Thrown when a user attempts an action that is not permitted for their role.

Key Responsibilities:

- Coordinate task operations and enforce business rules.
- Track task dependencies and maintain task history.
- Manage task state transitions and user permissions.
- Handle exceptions and ensure system integrity.
- Support persistence, reporting, and dashboard generation.

## 3.3 System Components and User Roles

The following describes the main system components and user roles within STRMS, along with their responsibilities and interactions:

- **Task**: Represents an individual unit of work (i.e., an individual task). It encapsulates its own information and behavior and maintains a reference to its associated **TaskHistoryEntry**. A task may depend on other tasks, meaning it cannot proceed until all its prerequisite tasks are completed. The `TaskManager` handles these dependencies, along with rules and lifecycle validation.
- **TaskHistoryEntry**: Maintains a record of all modifications made to a specific task, including what was changed, when the change occurred, and who performed it.
- **TaskManager**:
  - Serves as the central controller of the application.
  - Enforces all business rules related to task creation, deletion, assignment, state transitions, and dependency validation. Although users initiate actions, the `TaskManager` is responsible for executing them and maintaining system consistency.
  - Manages tasks, including adding, removing, searching, and listing them.
  - Operates independently of users and does not rely on user actions to function.
  - Assigns tasks while enforcing permissions, verifying that each user has the required role to perform the operation.
  - Consults user information to validate permissions and ensure that role-based constraints are respected.
- **User**: An abstract superclass representing all user roles, encapsulating common attributes and behaviors. The system defines three roles, **Admin**, **Manager**, and **Engineer**, which inherit from this class. All user roles interact with the `TaskManager` to perform their authorized operations as follows:

- **Admin**: Has full privileges, including creating, deleting, and assigning tasks via the `TaskManager`, as well as generating reports. For example, when a task creation request is made, the system verifies the role and permissions of the requesting user before adding the task. In other words, before performing any operation, the `TaskManager` ensures that the user possesses the required authorization.
- **Manager**: Responsible for assigning tasks to engineers, monitoring task progress, and generating reports.
- **Engineer**: Responsible for executing assigned tasks and updating their status. Engineers can only view, work on, and complete tasks assigned to them. Only engineers can execute tasks, meaning they are the only users allowed to work on and complete them.

Thus, these three subclasses represent distinct roles in the system, each with specific responsibilities and access permissions. All interactions between users and tasks are mediated by the `TaskManager`, which serves as the central controller of the system.

**Example Scenario** — A sample task workflow is described below —

**Actors**

- **TaskManager**
- **Admin (Alice)**
- **Manager (Bob)**
- **Engineer (Charlie)**

**Tasks**

- **Task1**: Implement login page
- **Task2**: Create database schema

**User and System Interactions**

- Alice creates **Task1** and **Task2** in the system. Both tasks are initially assigned the status `TODO`.
- The system verifies that Alice has permission to create tasks.
- Alice defines a dependency such that **Task2** depends on the completion of **Task1**.
- **Task2** is now `BLOCKED` because it depends on the completion of **Task1**.
- Bob assigns **Task1** to Charlie.
- The system verifies that Bob has permission to assign tasks.
- Charlie starts working on **Task1** and updates its status to `IN_PROGRESS`.
- Charlie completes **Task1** and updates its status to `DONE`.
- The system now allows **Task2** to be executed since its dependency is satisfied.
- Bob assigns **Task2** to Charlie.
- Charlie starts working on **Task2**, changing its status from `BLOCKED` to `IN_PROGRESS`.
- Alice deletes **Task1** after confirming that it has been completed.
- All actions are recorded in the task history.

A task history entry records all changes made to each task, including status updates, deadline modifications, the user who performed the change, etc.

# 4 Class Structure

## 4.1 User Classes

- **User (Abstract Class)**: Defines common properties and behaviors shared by all user types.
  - **Example of Attributes:** `id`, `name`, `email`, etc.
  - **Example of Abstract Methods:**
    * `canCreateTask()`: has the role/permission to create a task (implementation varies by subclass).
    * `canDeleteTask()`: has the role/permission to delete a task (implementation varies by subclass).
      etc.
- **Subclasses:** `Admin`, `Manager`, `Engineer`

## 4.2 Task Related Classes

- **Task:** Represents a single task in the system with history and dependencies. It holds its own data as well as the operations that act on those data. It maintains references to its prerequisite tasks. This class also allows tasks to be compared based on their priority so that higher-priority tasks are handled before lower-priority ones while respecting dependency rules.
  - Example of attributes: task id (unique), title, description, priority level, task status, task category, deadline, dependencies (storing the list of tasks on which the current task depends), history (representing a list of history entries), engineer (representing the assigned engineer), etc.
  - Example of methods: updateStatus, addHistoryEntry, displayTask, markAsDone, changePriority, updateDescription, etc.
- **TaskManager:** responsible for managing a collection of tasks. It represents the controller responsible for managing all tasks in the system. It manages adding, removing, searching, saving, and listing tasks, among others.
  - Example of attributes: The `TaskManager` class maintains a collection of all tasks using a *HashMap*, organizes ready-to-process tasks in a *PriorityQueue* based on priority, tracks tasks currently in progress using a *HashSet*, etc.
  - Example of methods:
    * **addTask**: Verifies that the user has permission to create a task, adds the task to the system, places it in the priority queue, and records a history entry indicating the creator.
    * **deleteTask**: Removes a task from the system only if it exists and the user has the necessary permissions.
    * **completeTask**: Allows the assigned engineer to mark a task as completed after verifying that the task exists, the user is the assigned engineer, and all dependencies are completed. The task is then removed from the tracking structures, and any tasks that depend on it may become eligible for execution.
    * **addDependency**: Adds a dependency between two tasks while ensuring the tasks exist, preventing a task from depending on itself, and avoiding circular dependencies.
    * **assignTask**: Assigns a task to an engineer if the user has the required permissions and the task is not already assigned. The task is then marked as in progress and added to the ready queue once all its dependencies are satisfied and completed.
    * **detectCircularDependency**: Uses a depth-first search (DFS) approach to check whether adding a dependency between tasks would create a circular dependency.
    * **printInProgressTasks**: Displays all tasks that are currently in progress (i.e., being worked on).
    * **findTask**: Searches for a task in the system using its identifier.
    * **saveTasksToFile**: Saves all tasks to a file for later reference.
    * **loadTasksFromFile**: Loads all tasks from a file into the system.
    * **updateTask**: Updates the state or attributes of an existing task.
      etc.

These are examples of attributes and methods to include. However, additional attributes and methods may be defined with different names or signatures, as long as they adhere to the same underlying concepts and maintain the core functionality and design principles.

- **TaskHistoryEntry:** It is neither a task nor a task manager. Its role is to record the history of a given task over time. It ensures traceability by tracking what changed, when the change occurred, who made the change, and other relevant details. Thus, it records the progress of a single task.
  Each history entry is associated with exactly one task and records the changes made to that task for auditing and traceability purposes.
  - Example of attributes: **action**: Stores the action performed on a task (e.g., creation, assignment, completion, etc.); **performedBy**: Indicates the user who performed the action; **timestamp**: Records the date and time when the action occurred, etc.

## 4.3 Enumerations

In java, enumerations are used to represent a finite and well-defined set of constant values. They improve code readability, ensure type safety, and prevent the use of invalid values.

The following enumerations must be defined and used consistently throughout the project:

### 4.3.1 TaskStatus

The `TaskStatus` enumeration represents the current state of a task within the task management system.

- **TODO**: The task has been created but work has not started yet.
- **BLOCKED**: The task cannot proceed due to unresolved dependencies or external constraints. The task remains in this state until all required dependencies are completed.
- **IN_PROGRESS**: The task is currently being worked on by an engineer. All dependencies must be in the **DONE** state before the task can enter this state.
- **DONE**: The task has been completed successfully and requires no further action. This is a terminal state: once a task reaches **DONE**, it cannot transition back to any previous state. If additional work related to the completed task is required, a new task must be created instead.

**State-transition diagram:**

$$\texttt{TODO} \rightarrow \texttt{BLOCKED} \rightarrow \texttt{IN_PROGRESS} \rightarrow \texttt{DONE}$$

This enumeration is essential for tracking tasks and generating reports on their progress.

### 4.3.2 PriorityLevel

The `PriorityLevel` enumeration defines the urgency and importance of a task.

- **LOW**: The task has minimal impact and can be completed at any time.
- **MEDIUM**: The task has normal priority and should be completed within a reasonable timeframe.
- **HIGH**: The task is important and should be addressed as soon as possible.
- **CRITICAL**: The task is urgent and must be handled immediately to avoid major issues.

This enumeration must be applied in task scheduling and priority-based task selection mechanisms.

### 4.3.3 TaskCategory

The `TaskCategory` enumeration classifies tasks according to their nature.

- **BUGFIX**: Tasks related to fixing defects or correcting errors in the system.
- **FEATURE**: Tasks involving the implementation of new functionalities.
- **DOCUMENTATION**: Tasks focused on writing or updating technical or user documentation.
- **RESEARCH**: Tasks requiring investigation, experimentation, or feasibility analysis.

This enumeration must be used for task classification, filtering, and reporting purposes.

### 4.3.4 NotificationType

The `NotificationType` enumeration specifies the method used to notify users about system events.

- **EMAIL**: Notifications sent via electronic mail.
- **SMS**: Notifications sent via short message service.
- **CONSOLE**: Notifications displayed directly in the application console.

This enumeration must be used in the notification module to determine how messages are delivered to users.

Enumeration Best Practices for Object-Oriented Design:
Proper use of enumerations will be evaluated as part of the code quality and object-oriented design criteria:

- Enumerations must not contain business logic.
- Enumerations must be used instead of string or integer constants.
- Enumerations must be referenced in UML diagrams where applicable.

## 4.4 Utility Classes

Students are requested to implement the following utility classes in Java according to their described responsibilities:

- **FileManager**: Provides methods to read from and write to files.
- **ReportGenerator (implements Reportable)**: Generates reports for tasks and users. Define a `Reportable` interface with a `generateReport()` method.
- **NotificationManager**: Provides a method to send notifications to users when tasks are updated or due.
- **Dashboard**: Displays statistics such as tasks per user, tasks per status, and overdue tasks (console output is sufficient).

You can start with minimal skeletons for each class and expand the functionality later. Focus on proper class design and method stubs.

# 5 Data Structures

Proper selection and use of data structures is crucial for efficient and maintainable code in STRMS. You are expected to apply appropriate data structures for different components of the project.
Table 1 provides a sample of data structures recommended for this project, along with their typical usage. Other structures with similar functionality are also acceptable.

| Data Structure | Purpose and Usage |
|---|---|
| `ArrayList<TaskHistoryEntry>` | Maintains a list of a task's history entries. |
| `HashMap<String, Task>` | Store all tasks |
| `HashMap<String, User>` | Enable quick lookup of users by ID |
| `HashSet<Task>` | Track tasks in progress |
| `PriorityQueue<Task>` | Manage tasks based on priority |
| *etc.* | ... |

Table 1: Sample data structures and their usage in the system

## 5.1 ArrayList

- **Use Cases:**
  - `ArrayList<Task>`: Maintaining and iterating over all tasks in the system.
  - `ArrayList<TaskHistoryEntry>`: Storing and iterating over the chronological history of each task.
- **Purpose:** To store objects in a dynamic, ordered collection that allows fast access by index.

## 5.2 HashMap

- **Use Cases:**
  - `HashMap<String, Task>`: Retrieving and managing tasks using their unique task identifiers.
  - `HashMap<String, User>`: Retrieving and managing users using identifiers such as user ID or email.
- **Purpose:** To map unique identifiers represented as `String` keys to their corresponding objects for fast lookup.

## 5.3 HashSet

- **Use Cases:**
  - `HashSet<Task>` to:
    * Track tasks that are currently being worked on.
    * Ensure that each task appears only once in the in-progress set.
    * Quickly check whether a task is already in progress.
    * Maintain a collection of tasks currently in progress without duplicates.

## 5.4 PriorityQueue

- **Use Cases:**
  - `PriorityQueue<Task>` to:
    * Support task scheduling for users.
    * Automatically manage and select the next task to execute according to priority levels.

**Guidelines:**

- Choose the most appropriate data structure based on access patterns, insertion and deletion frequency, and operation complexity.
- Maintain a clear separation between different collections (e.g., task list, user map, history list) to reduce coupling.
- Document the rationale for selecting each data structure in the project report.
- Ensure that data structures integrate properly with exception handling, task dependencies, and persistence mechanisms.
- Other Java collection data structures may be used when appropriate, as long as they follow the same design principles and their selection is clearly justified in the project report.

# 6 Task Dependencies and History

This section describes the mechanisms used to manage task dependencies and to maintain a history of task-related events. These features are essential for modeling real-world project constraints and for ensuring traceability and accountability within the system.

## 6.1 Task Dependencies

A task may depend on one or more other tasks. A dependency indicates that a task cannot start until all tasks it depends on (its prerequisite tasks) have been completed.

### 6.1.1 Dependency Rules

The following rules must be enforced by the system:

- Dependencies must be defined explicitly and stored as part of the task data.
- A task can move from `TODO` to `IN_PROGRESS` only if **all its dependencies are in the `DONE` state**.
- A task remains in `BLOCKED` if **any dependency is not `DONE`**. In other words, if at least one dependency is not completed, the task must remain in the `BLOCKED` state.

### 6.1.2 Circular Dependency Detection

The system must prevent the creation of **circular dependencies**, which would result in tasks that can never be completed.

- Circular dependency occurs when a task depends directly or indirectly on itself.
- Before adding a dependency, the system must verify that no cycle is introduced.
- If a circular dependency is detected, the operation must be rejected, and an appropriate exception must be raised.

This requirement encourages students to apply data structure concepts such as graphs, depth-first search, or dependency traversal to detect and prevent cycles.

### 6.1.3 Dependency-Based Task Activation

Before a task can be started, the system must perform a dependency validation:

- Priority-based task management must take task dependencies into account. In other words, task priorities must respect dependency constraints. Tasks are processed according to their urgency, but only after all their prerequisite tasks have been completed. For example, if Task B depends on Task A, Task A must be completed first, even if Task B has a higher priority.
- The system must verify the completion status of all prerequisite tasks.
- A task must not start if any of its prerequisite tasks are incomplete.
- An informative message or exception must be generated to explain why the task cannot proceed.

Students must design and implement mechanisms that allow:

- Adding one or more dependencies to a task after its creation.
- Removing dependencies when project requirements change.
- Listing all prerequisite tasks of a given task for visualization and reporting purposes.
- Validating that added dependencies do not create circular dependencies.
- Automatically updating a task's status when dependencies are added, removed, or when prerequisite tasks are marked as DONE.
- Checking whether a task is blocked or ready to start based on the completion status of its prerequisite tasks.
- Persisting dependency information to ensure data consistency across sessions.

## 6.2 Task History Tracking

In addition to managing dependencies, the system must maintain a detailed history of task-related events. This history ensures traceability and supports auditing of all actions performed on tasks.

### 6.2.1 Logged Events

A `TaskHistoryEntry` must record all significant task-related events (i.e., actions), including:

- Task creation (date and creator).
- Status changes (e.g., `TODO` → `IN_PROGRESS`, `IN_PROGRESS` → `DONE`).
- User assignment and reassignment.
- Dependency additions or removals.
- Task blocking or unblocking.

Each `TaskHistoryEntry` must include at minimum:

- A timestamp of when the event occurred.
- The user responsible for the action.
- A textual description of the event.

### 6.2.2 History Storage and Access

Students must ensure that:

- The system persistently stores the task history and restores it at application startup or whenever needed.
- The history entries are maintained in chronological order.
- The history of a task can be displayed or exported for reporting purposes.

### 6.2.3 Design Guidelines and Evaluation Criteria

- Keep the history mechanism separate from the core task business logic.
- History entries are immutable and read-only after being recorded.
- Ensure that dependency management prevents circular dependencies and correctly enforces task execution order.
- Validate all user actions to prevent invalid state changes in tasks or dependencies.
- Evaluate the quality of dependency management and history tracking based on correctness, robustness, maintainability, and clarity of design.
- Design the system for scalability to handle a growing number of tasks and users without degrading performance.

Proper implementation of task dependencies and history tracking is a key indicator of a student's ability to design complex object-oriented systems that incorporate data structures, enforce constraints, and ensure traceability.

## 6.3 Task Dependencies & Circular Dependency Detection

This subsection presents examples of dependency rules, valid task dependencies, and circular dependency detection to clarify the expected behavior of the system.

### 6.3.1 Example of Linear and Circular Dependencies

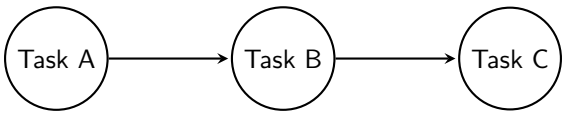
Consider the following tasks within the project:

- **Task A**: Design Database Schema
- **Task B**: Implement User Authentication
- **Task C**: Integrate Authentication with the Database

The dependencies are initially defined as follows:

- Task B depends on Task A (In other words, Task B cannot start or be completed until Task A has been completed. Hence, Task A must be finished first because Task B requires the results, output, or conditions of Task A to proceed.)
- Task C depends on Task B: Hence, A before B, and B before C, which implies A before C indirectly.

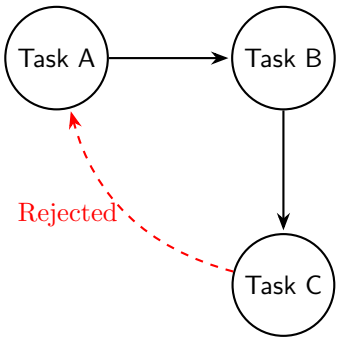
The current dependency graph is valid, representing a **linear** sequence of tasks. This results in the following dependency chain:




Now assume that a user attempts to add the following dependency:

- Task A depends on Task C

The resulting dependency chain becomes:




This situation creates a **circular dependency**. None of the involved tasks can be started because each task is waiting for another task in the cycle to be completed.

Attempt to Add a Circular Dependency (C → A):

- **Original graph:** Task A → Task B → Task C forms a valid acyclic graph.
- **After attempted addition:** Adding C → A would create a cycle. The red dashed bow labeled Rejected shows that the system denies it, keeping the graph consistent.

**Expected System Behavior:** When the dependency *Task A depends on Task C* is added, the system must:

- Detect that Task C is already indirectly dependent on Task A, thereby identifying the circular dependency.
- Reject the circular dependency addition.
- Inform the user that the addition operation would create a circular dependency.

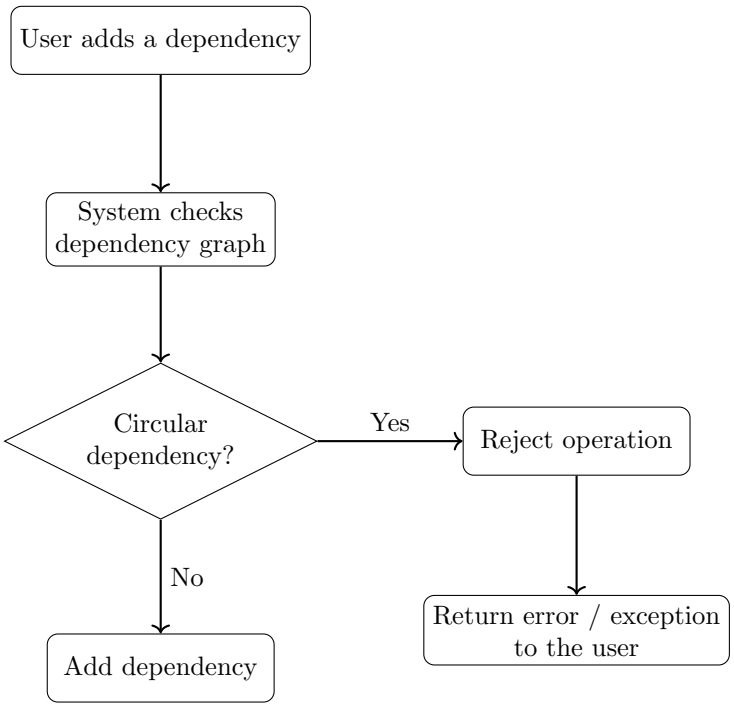
This validation must occur before the dependency is permanently stored. Consequently, circular dependencies must be detected before a new dependency is added. If a circular dependency is identified, the operation is rejected and the system state remains unchanged.

**Circular Dependency Prevention Rule:** To ensure system consistency and prevent task deadlocks, circular dependencies are strictly forbidden. Therefore, before adding any new dependency, the system must detect circular dependencies and prevent their creation:

- The system must detect circular dependencies **before** adding a new dependency.
- If a new dependency would create a circular dependency, the operation must be rejected.
- Rejected dependencies must **not** alter the existing system state.
- An appropriate exception or error message must be generated to notify the user.

At no point should the system contain circular task dependencies.

**Circular Dependency Detection Workflow:**




### 6.3.2 Example of Valid Task Dependency Scenario

Consider the following tasks in a project lifecycle:

- **Task 1**: Define Project Requirements
- **Task 2**: Design System Architecture
- **Task 3**: Implement Core Modules
- **Task 4**: Perform System Testing

The dependencies are defined as:

- Task 2 depends on Task 1
- Task 3 depends on Task 2
- Task 4 depends on Task 3

This forms a valid dependency structure:

$$\text{Task 1} \rightarrow \text{Task 2} \rightarrow \text{Task 3} \rightarrow \text{Task 4}$$

**Task Execution Flow:**

1. Task 1 has no dependencies and can be started immediately.
2. Once Task 1 is completed and marked as `DONE`, Task 2 becomes eligible to start.
3. Task 2 must be completed before Task 3 can begin.
4. Task 3 must be completed before Task 4 can begin; Task 4 remains in the `BLOCKED` state until then.

### 6.3.3 Example of Logging a Dependency Violation in History

Dependency Enforcement Scenario: We now consider the dependency graph illustrated in 6.3.2 in the following situation:

- A user attempts to start Task 3 before Task 2 is completed (i.e., Task 2 is still in the `IN_PROGRESS` state).
- The system detects that the required dependency is not satisfied.
- The system rejects the action and keeps Task 3 in the `BLOCKED` state.

- The system provides a clear explanation indicating which dependency is incomplete.
- A history entry is recorded for Task 3.

Logical flow: attempt → detection → rejection → logging.

The history entry must include the following information:

- The timestamp of the attempted action.
- The user who attempted the action.
- The action that was attempted.
- The reason for the rejection.

This ensures traceability and accountability for every task-related action and decision made by the system.

# 7 JUnit Testing Requirements

Students are required to validate dependency-related behavior through automated unit tests using `JUnit`. The `JUnit` test suite interacts with the main system components to verify correct behavior, ensuring that both normal operations and exceptional scenarios are properly handled.

## 7.1 JUnit Test Suite

This is the automated testing component of the system, responsible for verifying task execution, state transitions, task lifecycle operations, and the behavior of the `TaskManager`. The `JUnit` test suite verifies all major components, covering both normal execution scenarios and expected failure cases that trigger exceptions. It ensures the correct operation of the system by:

- Ensuring task dependency rules are respected.
- Detecting circular dependencies.
- Verifying task priorities and state transitions.
- Validating user role permissions.

## 7.2 Mandatory Test Scenarios

The test suite must include, at a minimum, the following scenarios:

- **Successful addition of valid dependencies:** Ensure that tasks can be linked correctly when no cycles are introduced.
- **Rejection of circular dependencies:** Any attempt to add a dependency that would create a cycle should be denied, preserving the acyclic nature of the task graph.
- **Maintenance of graph integrity after rejection:** By rejecting a dependency that would form a cycle, the graph remains consistent. All previously valid dependencies are preserved, and the graph remains suitable for scheduling.
- **Exception or error handling:** Detecting a circular dependency should trigger a clear exception or error to inform the user or system of the invalid operation.
- **Correct removal of dependencies:** Removing a dependency should update the task graph appropriately and adjust task states as needed, without introducing inconsistencies.

## 7.3 Testing Constraints

- Tests must be isolated and repeatable, ensuring consistent results regardless of execution order.
- Validation of test outcomes should be fully automated, requiring no manual intervention.
- Dependency-related tests must be clearly documented, with descriptive names reflecting their purpose.

The correctness and completeness of dependency-related unit tests will be a primary evaluation criterion.

# 8 Custom Exceptions

This project requires implementing robust exception handling to maintain system integrity and handle task-specific errors effectively. The following list provides examples of custom exceptions to be implemented in the system, as previously outlined in TaskManager:

- **CircularDependencyException**
  - Thrown when a new dependency would create a circular dependency among tasks.
  - Ensures that tasks cannot form deadlocks.
  - Must prevent any changes to the task dependency graph.
- **DependencyNotCompletedException**
  - Thrown when a user attempts to start or progress a task whose dependencies are not yet completed.
  - Enforces proper execution order of tasks.
- **TaskNotFoundException**
  - Thrown when an operation references a task ID that does not exist in the system.
  - Useful for validation in task assignment, deletion, and updates.
- **InvalidRoleException**
  - Thrown when a user attempts an action that is not permitted for their role (e.g., an engineer trying to assign tasks).
  - Ensures role-based access control.
- **FilePersistenceException**
  - Thrown when a read or write operation to a file fails (e.g., file not found, permission denied, corrupted file).
  - Ensures that data persistence operations are handled safely.
- **DuplicateTaskException**
  - Thrown when attempting to create or add a task with an ID that already exists in the system.
  - Prevents duplicate tasks and maintains data integrity.
- **InvalidTaskStateException**
  - Thrown when an invalid state transition is attempted (e.g., moving from `DONE` back to `IN_PROGRESS`).
  - Ensures proper task lifecycle management.

**Best Practices:**

- Each exception must be used where appropriate.
- Custom exceptions should be used when needed and must extend from Java's `Exception` hierarchy.
- All exception messages should be clear and descriptive to help users and testers understand the error.
- Exception handling must be integrated with both task operations and JUnit test cases, especially for circular dependency and invalid task actions.

# 9 UML Guidelines

UML diagrams provide a standard way to visualize system structure and behavior, clarify design decisions, highlight component dependencies, and support team collaboration. The following guidelines outline best practices for this project:

- Create class diagrams that include all classes, attributes, methods, and relationships.
- Develop sequence diagrams to represent task assignment and interactions.
- Use UML tools such as StarUML, Draw.io, or Lucidchart to create clear and consistent diagrams.

## 10 Implementation Phases

*Note: The following development plan is indicative and may vary depending on the project's progress.*

| | **Project Development Steps** |
|---|---|
| 1 | Requirements analysis and planning |
| 2 | UML diagrams: class and sequence diagrams |
| 3 | Core Java implementation (User, enums, Task, TaskManager, etc.) |
| 4 | File I/O, persistence, and exception handling |
| 5 | Advanced features: task dependencies, history, priority queue, notifications, etc. |
| 6 | JUnit testing (unit and integration), possibly performed incrementally during implementation |
| 7 | Report generation and dashboard: exporting summaries (e.g., text file, console report) |

## 11 Project Instructions

- Implement all methods yourself; focus on **design, logic, and OOP principles**.
- Ensure **task dependency rules** are enforced and **track task history** for every change.
- Handle exceptions properly for invalid operations.
- Write clean, well-documented, and readable code following standard **Java coding conventions**.
- Provide **UML diagrams** as part of the submission.
- Write **JUnit tests** for both core and advanced features.
- Use a version control system (e.g., Git) to collaborate efficiently and keep track of code changes during the project.
- **Weekly submissions:** Project progress must be uploaded to Moodle every week until the final project presentation. Submissions may include updated source code, UML diagrams, documentation, test cases, or any other material reflecting the team's progress.
- Submit a final report in **PDF format** including the project description, UML diagrams, and relevant screenshots.
- Submit the complete **project source code** (all Java files, test files, and resources) as part of the final submission.
- An oral examination will take place on June 4, during which you will present your work. You must prepare a short presentation for this purpose.
- **Team size:** This project is designed for a team of **4–5 students**. Teams larger than 5 are not recommended due to coordination and grading challenges.
- **Plagiarism and Copy-Paste:** All students must submit original work. Copying code or solutions from other students, online sources, or repositories is strictly forbidden.
- **Use of AI tools:** Using AI tools such as ChatGPT to generate code or solutions without proper understanding or documentation is considered academic dishonesty. Students may consult AI tools only for learning and debugging guidance, but the final implementation must be their own work.
- **Citation requirement:** If any external resources are referenced (books, articles, websites, tutorials), they must be properly cited in the report.
- **Consequences:** Violating these rules may result in a zero grade for the project and could lead to additional academic disciplinary actions in accordance with school policies.

  Reminder: Collaboration within the assigned team is mandatory. Team members should clearly divide responsibilities and maintain a shared understanding of the project architecture, following the recommended design patterns and coding practices.

**Good Luck and Happy Coding!**